Tugas Mandiri 3 - Kajian Content-based Filtering

Content-Based Filtering dengan Word Embedding dan Sentence Embedding

Laporan tugas mandiri

DISUSUN OLEH	NIM	TANGGAL
Najib Robbani	**103042580006**	**Oktober 2026**

1. Pendahuluan

Content-based filtering (CBF) adalah metode rekomendasi yang menyarankan item berdasarkan kemiripan antara item yang disukai pengguna dan item lain. Metode ini tidak membutuhkan data pengguna lain, sehingga cocok ketika item baru belum memiliki interaksi. Kualitas CBF sangat bergantung pada cara fitur item direpresentasikan. Pada teks, TF-IDF hanya melihat kata yang sama, sedangkan word embedding dan sentence embedding dapat menangkap kemiripan makna.

Tujuan tugas ini adalah:

- Menjelaskan konsep dasar content-based filtering.
- Menjelaskan cara kerja word embedding dan sentence embedding.
- Memberikan contoh sederhana penggunaan keduanya.
- Membandingkan hasil TF-IDF, word embedding, dan Sentence-BERT pada data yang sama.

2. Content-Based Filtering

2.1 Konsep

CBF membandingkan profil pengguna dengan profil item. Profil pengguna dibangun dari item yang disukai, misalnya dengan merata-ratakan vektor item tersebut. Skor rekomendasi dihitung dengan fungsi kemiripan:

$$score(u, i) = similarity(profile(u), features(i))$$

Semakin mirip profil pengguna dengan fitur item, semakin tinggi skor item tersebut.

2.2 TF-IDF dan Cosine Similarity

TF-IDF memberi bobot tinggi pada kata yang sering muncul di satu dokumen tetapi jarang muncul di dokumen lain.

$$tf\text{-}idf(t, d) = tf(t, d) \times idf(t), \text{ dengan } idf(t) = \ln((1 + N) / (1 + df(t))) + 1$$

Setelah dokumen menjadi vektor, kemiripannya diukur dengan cosine similarity:

$$cos(q, d) = (q \cdot d) / (||q|| \times ||d||)$$

Nilai cosine mendekati 1 berarti dua vektor searah. Nilai 0 berarti tidak ada kemiripan.

2.3 Keterbatasan TF-IDF

- Tidak mengenali sinonim dan parafrasa.
- Tidak membedakan makna kata yang sama pada konteks berbeda.
- Tidak mempertimbangkan urutan kata dan negasi.
- Vektor renggang dan kurang efektif pada teks pendek.

Karena itu, diperlukan representasi yang lebih semantik.

3. Word Embedding

3.1 Konsep

Word embedding adalah representasi kata sebagai vektor padat. Vektor dipelajari dari konteks: kata yang sering muncul di lingkungan yang sama akan memiliki vektor berdekatan. Contohnya, *astronaut* akan dekat dengan *space*, *Mars*, dan *survive*.

3.2 Metode Populer

- **Word2Vec** memprediksi kata dari konteksnya (CBOW) atau konteks dari kata (Skip-gram).
- **GloVe** memanfaatkan matriks co-occurrence global.
- **FastText** memakai subword, sehingga lebih tahan terhadap kata baru dan salah ketik.

Pada tugas ini, word embedding dibuat dengan cara sederhana: co-occurrence → PPMI → SVD. Tujuannya agar prosesnya mudah dijelaskan.

3.3 Contoh Sederhana

Program demo memakai 12 deskripsi film. Langkah pembuatan word embedding:

1. Tokenisasi dan penghapusan stopword.
2. Hitung co-occurrence dalam jendela dua kata.
3. Ubah counts menjadi PPMI.
4. Reduksi dimensi dengan SVD menjadi 48 dimensi.
5. Normalisasi L2.

Hasilnya adalah 92 kata kosakata dan matriks 92 × 92. Beberapa tetangga terdekat:

Kata	Tetangga terdekat (cosine)
space	shuttle (0,56), destroy (0,42), ranger (0,34), orbit (0,34)
astronaut	survive (0,45), fight (0,45), mars (0,43), stranded (0,43)
love	fall (0,67), young (0,58), test (0,38), artist (0,37)
crime	patriarch (0,75), aging (0,52), control (0,49), family (0,42)

Tabel 1. Contoh tetangga kata pada word embedding demo.

Hasil ini menunjukkan bahwa word embedding dapat menangkap kedekatan makna. Namun, kata *space* juga dekat dengan *ranger* karena *Toy Story* memuat frasa “space ranger”. Ini menunjukkan keterbatasan embedding statis yang hanya memberi satu vektor per kata.

3.4 Penggunaan dalam CBF

Vektor kata digabungkan menjadi vektor dokumen dengan mean pooling:

$$d = (1 / |T|) \times \Sigma\, v_w$$

Setelah itu, cosine similarity dihitung antara vektor query dan vektor item.

```
def mean_sentence_vector(text, word_vectors):
    words = [w for w in tokenize(text) if w in word_vectors]
    if not words:
        return None
    vector = np.mean([word_vectors[w] for w in words], axis=0)
    return vector / np.linalg.norm(vector)
```

Contoh: query *Interstellar* dan item *The Martian* memiliki cosine 0,5225. Dengan TF-IDF, pasangan yang sama hanya mendapat 0,1743. Word embedding lebih tinggi karena memanfaatkan kedekatan makna kata.

3.5 Kelebihan dan Kekurangan

Kelebihan	Kekurangan
Menangkap kemiripan makna dan variasi kata.	Mean pooling menghilangkan urutan kata.
Vektor padat dan relatif ringan.	Satu kata hanya punya satu vektor.
Mudah dipakai untuk teks pendek.	Kata di luar kosakata tidak dikenali.

4. Sentence Embedding

4.1 Konsep

Sentence embedding memetakan satu kalimat menjadi satu vektor. Berbeda dari mean pooling, model sentence embedding mempertimbangkan konteks dan urutan kata. Salah satu model yang populer adalah Sentence-BERT (SBERT).

4.2 Mean Pooling dan Sentence-BERT

Mean pooling adalah cara paling sederhana: vektor kalimat adalah rata-rata vektor kata. Cara ini murah, tetapi belum memahami komposisi kalimat.

Sentence-BERT menggunakan encoder transformer yang dilatih agar kalimat dengan makna mirip memiliki vektor mirip. Prosesnya:

1. Kalimat dimasukkan ke encoder.
2. Hasil token di-pooling menjadi satu vektor.
3. Vektor dinormalisasi.
4. Kemiripan dihitung dengan cosine similarity.

4.3 Contoh Sederhana

Pada demo, 12 deskripsi film diubah menjadi vektor 384 dimensi dengan model `all-MiniLM-L6-v2`. Hasil rekomendasi untuk query *Interstellar*:

Peringkat	Item	Skor
1	The Martian	0,4770
2	Gravity	0,3277
3	Arrival	0,3219
4	La La Land	0,1371
5	Toy Story	0,1088

Tabel 2. Rekomendasi Sentence-BERT untuk query *Interstellar*.

SBERT menempatkan *Arrival* di atas *Toy Story*. *Arrival* memang tidak banyak berbagi kata dengan *Interstellar*, tetapi temanya dekat: alien, bahasa, dan kontak pertama. *Toy Story* turun karena “space ranger” adalah mainan, bukan perjalanan antariksa.

4.4 Penggunaan dalam CBF

```
from sentence_transformers import SentenceTransformer

model = SentenceTransformer("sentence-transformers/all-MiniLM-L6-v2")
item_vectors = model.encode(descriptions, normalize_embeddings=True)
scores = item_vectors @ user_vector
```

Vektor item dihitung sekali, lalu disimpan. Saat pengguna menyukai item baru, profil pengguna diperbarui dengan merata-ratakan vektor item yang disukai.

4.5 Kelebihan dan Kekurangan

Kelebihan	Kekurangan
Memahami makna dan konteks kalimat.	Lebih berat daripada TF-IDF.
Lebih tahan terhadap parafrasa.	Sulit dijelaskan secara rinci.
Tersedia model multibahasa.	Kualitas bergantung pada domain model.

5. Contoh dan Perbandingan

5.1 Hasil Rekomendasi

Query: *Interstellar*. Top-3 untuk setiap metode:

Metode	Peringkat 1	Peringkat 2	Peringkat 3
TF-IDF	The Martian (0,174)	Gravity (0,142)	Toy Story (0,141)
Word Embedding	The Martian (0,523)	Gravity (0,486)	Toy Story (0,369)
Sentence-BERT	The Martian (0,477)	Gravity (0,328)	Arrival (0,322)

Tabel 3. Perbandingan top-3 rekomendasi.

5.2 Analisis Singkat

Ketiga metode sepakat bahwa *The Martian* adalah rekomendasi terbaik. Namun, TF-IDF dan word embedding menempatkan *Toy Story* di peringkat ketiga karena sama-sama memiliki kata “space”. SBERT tidak tertipu karena memahami konteks “space ranger”.

Word embedding sudah lebih baik daripada TF-IDF dalam menangkap kedekatan kata, tetapi mean pooling masih kehilangan urutan dan konteks. SBERT memberi peringkat yang paling masuk akal secara semantik, dengan biaya komputasi lebih tinggi.

5.3 Kapan Menggunakan

Kebutuhan	Pilihan
Baseline cepat dan mudah dijelaskan	TF-IDF
Kemiripan makna kata dengan biaya rendah	Word embedding
Kualitas semantik yang lebih tinggi	Sentence-BERT
Sistem produksi	Gabungkan filter kata kunci dan skor semantik

6. Kesimpulan

Content-based filtering merekomendasikan item berdasarkan kemiripan fitur. TF-IDF cepat dan mudah dijelaskan, tetapi hanya memahami kata yang sama. Word embedding menambahkan pemahaman makna kata, sedangkan sentence embedding memahami makna kalimat secara lebih utuh.

Pada contoh 12 film, semua metode memilih *The Martian* untuk query *Interstellar*. Perbedaannya terlihat pada peringkat berikutnya: TF-IDF dan word embedding masih terpengaruh kata “space” pada *Toy Story*, sedangkan Sentence-BERT memilih *Arrival* karena kedekatan makna.

Untuk tugas sederhana, TF-IDF atau word embedding sudah cukup. Jika kualitas semantik lebih penting dan sumber daya tersedia, Sentence-BERT adalah pilihan yang lebih baik.

Lampiran: Program

Program demo ada di folder `program/`. Untuk menjalankan:

```
pip install numpy scikit-learn
python3 cbf_embeddings.py
```

Untuk membandingkan dengan Sentence-BERT:

```
pip install sentence-transformers
```

Program akan menghasilkan `assets/demo-data.json`, `assets/demo-data.js`, dan `output.txt`. Ringkasan output:

```
Jumlah item            : 12 film
Kosakata               : 92 kata
Dimensi word embedding : 48
Model Sentence-BERT    : all-MiniLM-L6-v2 (384 dimensi)

Query: Interstellar
TF-IDF         : The Martian 0.1743, Gravity 0.1417, Toy Story 0.1414
Word Embedding : The Martian 0.5225, Gravity 0.4860, Toy Story 0.3690
Sentence-BERT  : The Martian 0.4770, Gravity 0.3277, Arrival 0.3219
```

Demo interaktif: https://najibrobbani74.github.io/college-task/content-based-filtering/